

V3 Implementation Document

Data Contract, Tier 0 Pre-Checks, and the Tier 1 Pipeline
for Online Thought-Anchor Detection

Build specification — companion to the V3 design report

Scope and status. This is a *build* document; the design rationale lives in the V3 report and is referenced, not restated. It is decision-complete for two things only: the data contract (Part A, designed against every tier) and the Tier 0/Tier 1 pipeline (Parts B–D). Later tiers are deliberately left as typed stubs (Part E): their logic is not written, because which branch is needed is not known until Tier 1’s diagnostics return. Blue boxes mark *made choices* the implementer may override; red boxes mark *hazards*; the green box is a hard *gate*. Traces are *self-generated* (not borrowed), so the schema is designed from scratch against the target model and the future ∇V probe’s needs.

Contents

1	Orientation: what is built when, and why in this order	2
2	Part A — Shared data contract and storage schema	2
2.1	Index and granularity	2
2.2	Stored fields	2
2.3	The stride decision (anticipating the probe)	3
3	Part B — Tier 0: cheap pre-checks before any labeling	3
3.1	Bucket 1 — plumbing smoke tests (does the machinery work)	3
3.2	Bucket 2 — assumption probes (is the premise plausible)	4
3.3	Bucket 3 — calibration measurements (cheap inputs Tier 1 needs)	4
4	Part C — Go / no-go gate	5
5	Part D — Tier 1: the minimal labeling pipeline and detector	5
5.1	Parameter table (first-run defaults and sweep ranges)	5
5.2	Stage 1+2 — segmentation (deterministic rule)	6
5.3	Stage 3 — labels via perturbed resampling	6
5.4	Answer extraction (per-task rule)	7
5.5	Stage 4 — physics signals (the detector)	7
5.6	The diagnostic and the two detectors	7
5.7	Evaluation against labels and baselines	7
6	Part E — Later tiers (typed stubs)	8

1 Orientation: what is built when, and why in this order

The document runs in the order of increasing commitment cost, so it reads as it executes. Part A fixes the on-disk *data contract* once, against the union of all tiers’ needs, because regenerating an archive after a schema change is the dominant avoidable cost. Part B (Tier 0) runs cheap checks on a handful of traces that can *kill or redirect* the project before any labeling compute is spent; it emits *named outputs*. Part C is a hard go/no-go gate. Part D (Tier 1) implements the first real detector and labeling run, consuming Tier 0’s named outputs by reference. Part E stubs later tiers.

Governing rule: plumb the storage and interfaces for the union of all tiers; implement the logic for only the current tier. The data contract (Part A) anticipates the ∇V probe and every branch; the *code* implements only Tier 0 and Tier 1.

2 Part A — Shared data contract and storage schema

Every tier reads from one archive with a single index. The schema is fixed now and treated as an invariant; later tiers add *consumers*, never migrations.

2.1 Index and granularity

The primary key is (item, trace, run, layer, token):

- **item:** a task instance (one problem). Carries the gold answer and any constructive on-path/off-path fact annotations (§3.3).
- **trace:** one base generation for an item (the “real” chain of thought).
- **run:** a rollout index. **run=0** is the base trace; **run>0** are real or perturbed continuations used for labels (§5.3).
- **layer:** the residual-stream layer index a hidden state was read from.
- **token:** absolute token position within the (prompt + generation) sequence.

The independent statistical unit is the **trace**, never the span (within-trace spans are correlated).

2.2 Stored fields

Field	Content and rationale
<code>token_ids</code>	The <i>generated</i> token ids (prompt and generation separated). Never re-tokenized text — see hazard below.
<code>char_offsets</code>	Per-token character spans, so a token window maps back to text.
<code>clause_flags</code>	Per-token Stage-1 clause-boundary flags.
<code>window_flags</code>	Per-token Stage-2 window-boundary flags (the span edges).
<code>surprisal</code>	Per-token $-\log P_\theta(\text{tok})$, for the potential V . Stored at generation time.
<code>hidden</code>	Hidden states h_t , stored at layer set <code>layers_stored</code> and token stride <code>store_stride</code> . Sized for the <i>future</i> ∇V probe, not just Tier 1 (see §2.3).

<code>logits_topk</code>	Top- K_{store} output logprobs per stored token (for V as a field and the KL proxy), plus tail log-mass.
<code>answer_dist</code>	Per-run final-answer distribution (the extracted discrete answer; §5.3).
<code>A_i</code>	Per-span counterfactual-importance label, once computed.

The alignment invariant (load-bearing). Re-tokenizing stored *text* can yield a different token count than generation produced (whitespace, special tokens, BPE merges at span edges), silently shifting every per-token signal relative to its label. The schema therefore stores `token_ids` as the source of truth; text is derived, never re-tokenized for alignment. The Tier 0 check `alignment_ok` (§3.1) verifies this on real traces before anything depends on it.

2.3 The stride decision (anticipating the probe)

`store_stride` and `layers_stored` are fixed *now* against the most demanding future consumer — the off-path ∇V probe (V3 report § “potential and its gradient”) — not against Tier 1, which uses far less.

Choice (override if profiling dictates): store `hidden` at *every* token (`store_stride=1`) for the Tier 0/1 trace set, at the single layer `layer_default` (Tier 0 output) *plus* the two adjacent layers, and store `logits_topk` with $K_{\text{store}} = 512$. Rationale: Tier 1 needs only h at span edges, but the probe (a fix/improvement tier) needs h at arbitrary positions and the upper-layer stack; storing densely on the *small* Tier 1 trace set is cheap and removes a regeneration if a probe tier is later entered. At large scale (Tier 3+), `store_stride` rises and only span-edge tokens are kept dense — a documented later change, not a migration of the Tier 1 archive.

This is the one place upfront over-provisioning is justified, because the alternative is regenerating traces. It is *not* licence to over-build elsewhere: only the storage is provisioned for all tiers; no probe code is written until a probe tier is entered.

3 Part B — Tier 0: cheap pre-checks before any labeling

Tier 0 runs on a handful of base traces (no rollouts, no labels, no detector). Each check is *procedure* $\rightarrow$ *cost* $\rightarrow$ *verdict* $\rightarrow$ *what-it-changes*, and the ones that produce values Tier 1 needs emit a **named output**. The asymmetry to keep in mind throughout: Tier 0 can *kill or redirect* the project, but passing it means “not obviously dead,” never “likely to work.”

3.1 Bucket 1 — plumbing smoke tests (does the machinery work)

`alignment_ok` — token-alignment integrity

Procedure: generate ~ 5 traces; store `token_ids`; independently re-tokenize the decoded text; compare. *Cost:* minutes. *Verdict:* pass iff counts and positions match exactly. *What-it-changes:* **project-ender if it fails** — every per-token signal is mislabeled otherwise. Fix the storage path before proceeding.

hidden-state sanity

Procedure: pull `hidden` at `layer_default` candidates; check shape, finiteness, and non-degeneracy

(not all-equal, not exploding). *Cost*: minutes. *Verdict*: pass iff finite and varying. *What-it-changes*: a fail means the extraction hook is wrong; fix before proceeding.

surprisal cross-check

Procedure: compare stored **surprisal** against the model’s own `log_softmax(logits)` at the realized tokens. *Cost*: minutes. *Verdict*: pass iff they match to numerical tolerance. *What-it-changes*: a fail means V is computed wrong; fix before proceeding.

segmenter sanity

Procedure: run the clause→window segmenter (§5.2) on real traces; read output by eye. *Cost*: an hour. *Verdict*: pass iff spans are well-formed (not all degenerate one-token spans; no mid-number/format splits). *What-it-changes*: a fail sends you back to the segmentation rule, not the model.

3.2 Bucket 2 — assumption probes (is the premise plausible)

artifact_verdict — velocity artifact domination

Procedure: compute $v_t = \Delta h_t$ on a few traces; inspect whether a couple of outlier dimensions or monotonic norm drift dominate the variance. *Cost*: an hour. *Verdict*: record whether raw velocity is artifact-dominated. *What-it-changes*: if dominated, the whitening metric is *load-bearing* (not optional) — which Tier 1 already uses, so this informs expectations, and a **severe** domination that whitening cannot remove is a redirect signal. Can kill, cannot validate.

energy computability

Procedure: compute $E_t = \frac{1}{2}v^\top Gv + V$ with a crude constant G on a few traces. *Cost*: an hour. *Verdict*: pass iff E is finite, varies, and neither term swamps the other by orders of magnitude. *What-it-changes*: a degenerate E (one term dominating) means the metric scale or V normalization needs fixing before Tier 1’s Test 1 is meaningful.

layer_default — layer prior

Procedure: eyeball trajectory structure across candidate layers (~ 0.5 – 0.85 depth). *Cost*: an hour. *Verdict*: pick an informed default (emit `layer_default`). *What-it-changes*: sets Tier 1’s layer sweep centre; not a kill check.

Bucket 2’s green lights are *weak*. Structured-looking velocity on five traces means the cheap precondition is not obviously violated — not that the dynamics carry anchor signal. Tier 0 can only remove the project from consideration; it cannot confirm it. Do not let a Bucket-2 pass become confidence.

3.3 Bucket 3 — calibration measurements (cheap inputs Tier 1 needs)

cosine_median — semantic-filter threshold

Procedure: embed all spans (Sentence-BERT) over the Tier-0 trace set; compute the median pairwise cosine over *clause-respecting spans* (§5.2). *Cost*: minutes once embeddings exist. *Verdict*: emit `cosine_median`. *What-it-changes*: this *is* the Tier-1 filter threshold; it must come from our own span distribution, never borrowed from the paper.

rejection_rate — re-draw feasibility

Procedure: on a few spans, resample i' and apply the `cosine_median` filter; measure the fraction

rejected. *Cost*: a few rollouts. *Verdict*: emit `rejection_rate`. *What-it-changes*: sets whether the Tier-1 re-draw budget (`redraw_budget`) is realistic; a very high rejection rate inflates the rollout cost and must be known before launch.

`compute_estimate` — `budget`

Procedure: from span count per trace, R , and `rejection_rate`, estimate total rollouts for the Tier-1 set. *Cost*: arithmetic. *Verdict*: emit `compute_estimate`. *What-it-changes*: go/no-go input; if infeasible, drop R to the pilot value or shrink the trace set before committing.

4 Part C — Go / no-go gate

Do not spend Tier 1 labeling compute until all of the following hold.

1. `alignment_ok` = pass. (**Project-ender if not.**)
2. hidden-state, surprisal, and segmenter smoke tests = pass.
3. `artifact_verdict` is not “severe, whitening-irremovable.” (**Redirect if it is.**)
4. energy computability = pass (no order-of-magnitude term domination).
5. `cosine_median`, `rejection_rate`, `compute_estimate` measured and recorded; `compute_estimate` within budget.
6. `layer_default` chosen.

Passing this gate means the machinery works and the premise is not obviously dead — not that Tier 1 will succeed. Cross it deliberately.

5 Part D — Tier 1: the minimal labeling pipeline and detector

Tier 1 is implemented in full. It consumes Tier 0’s named outputs by reference (`cosine_median`, `layer_default`, `rejection_rate`) and never restates their values. Its first job is the Test 1 diagnostic, which selects the detector (Branch A or B); see the V3 report for why this is diagnostic, not assumed.

5.1 Parameter table (first-run defaults and sweep ranges)

Every parameter the V3 report marked **sweep** is given a first-run default *and* a range here, because an implementer needs a concrete starting value. Defaults are the value used for the first Tier-1 run; ranges are for the subsequent sweep.

Parameter	First-run default	Sweep range	Notes
<code>layer</code>	<code>layer_default</code>	0.5–0.85 depth	centre from Tier 0.
<code>sg_halfwidth w</code>	4 tokens	2–8	forward half is the lag budget.
<code>sg_polyorder</code>	3	2–3	Savitzky–Golay order.
<code>tau</code> (window)	8 tokens	4–16	max tokens/window within a clause.
<code>shrinkage</code>	0.1 (Ledoit–Wolf auto)	auto, or 0.01–0.3	for Σ^{-1} .
<code>R_pilot</code>	16	—	rollouts/span, pilot.
<code>R_gold</code>	100	—	rollouts/span, gold labels.
<code>redraw_budget</code>	5 attempts	3–10	then drop span; check vs. <code>rejection_rate</code> .
<code>K_store</code>	512	—	stored top- K logprobs.
<code>temperature</code>	0.6	0.6–1.0	resampling temperature.

Choices made (override freely): `R_pilot`=16 balances label noise against a cheap first pass; `R_gold`=100 matches the original Thought-Anchors rollout count. `redraw_budget`=5 with drop-on-exhaustion prevents a hard-to-perturb span from stalling the pipeline; if `rejection_rate` is high, raise it or accept more dropped spans. `sg_halfwidth`=4 is a small lag; widen if velocity is noisy (Tier 0 `artifact_verdict` informs this).

5.2 Stage 1+2 — segmentation (deterministic rule)

Choice: clause split via a deterministic rule on the stored `token_ids/char_offsets`: break at sentence terminators (`.!?` followed by space/newline), newlines, commas, and a fixed connective list `{so, therefore, thus, then, because, but, and so, which means, hence}`; then window each clause into runs of $\leq \tau$ tokens (`tau` default 8, range 4–16), merging a trailing fragment $< \lceil \tau/2 \rceil$ into its neighbour *within the same clause*. Library: a dependency-light sentence splitter (e.g. `pysbd`) for sentences, then the rule above for clauses; do not use a parser that re-tokenizes.

Windows never cross a clause boundary (V3 report § granularity). Emit `clause_flags` and `window_flags` into the archive.

5.3 Stage 3 — labels via perturbed resampling

For each span i :

1. Generate R “real” continuations from the prefix through i ($R = R_pilot$ for the pilot run, R_gold for gold labels).
2. Resample candidate i' ; accept iff $\cos(i, i') < \text{cosine_median}$ (Sentence-BERT); re-draw up to `redraw_budget`, else drop the span. Generate R “perturbed” continuations from the prefix with $i \rightarrow i'$.
3. Extract the discrete answer from each continuation (§5.4); aggregate into `answer_dist` for real and perturbed.
4. Label $A_i = \text{TV}(\hat{p}_{\text{real}}, \hat{p}_{\text{pert}}) = \frac{1}{2} \sum_a |\hat{p}_{\text{real}}(a) - \hat{p}_{\text{pert}}(a)|$.

Adjudicator choice (per the design): the cosine-median filter is the headline adjudicator (scales unchanged, identical label definition across model sizes). NLI non-entailment is an *optional* fidelity cross-check on the small target model only, never mixed into the primary label set.

5.4 Answer extraction (per-task rule)

Choice: on the verifiable curriculum, extract the answer by a per-task regex/marker rule (boxed answer `\boxed{}` for math; final-line entity for state-tracking; SAT/boolean literal for satisfiability). A continuation with no extractable answer is recorded as a distinct “no-answer” outcome in `answer_dist` (not dropped), so A_i reflects answer-destruction too.

5.5 Stage 4 — physics signals (the detector)

On the same traces, compute (pure array work, no extra model passes beyond stored fields):

1. Velocity v_t : Savitzky–Golay (`sg_halfwidth`, `sg_polyorder`) on `hidden` at `layer`, per token, $dt = 1$.
2. Metric $G = \Sigma^{-1}$: Ledoit–Wolf (`shrinkage`) on velocity covariance pooled over a *held-out* trace split (no leakage).
3. Potential V : per-span summed `surprisal`.
4. Energy $E_t = \frac{1}{2}v^\top Gv + V$; rate dE/dt by centred difference (uses the lag).

5.6 The diagnostic and the two detectors

1. **Test 1 (selects the detector):** measure coasting dE/dt on the coasting set (off-path spans from the constructive curriculum where available, else low-surprisal runs). If ≈ 0 : **Branch A**, anchor score $|dE/dt|$. If structured decay: **Branch B**, fit drag C on coasting, anchor score $dE/dt + 2\mathcal{F}$.
2. **Test 3 (diagnostic):** recompute the score under arc-length time; flag if the anchor ranking changes materially (clock artifact $\rightarrow$ a fix tier).
3. **Test 2 deferred** (needs the ∇V probe; a Tier-2+ feature).

5.7 Evaluation against labels and baselines

Score per-span anchor signal against A_i with Spearman correlation (trace as the unit), and report against both baselines on the same traces/labels: the length-corrected kinematic signal, and the learned step-boundary classifier trained on A_i . The two collapse checks (V3 report § meaning): the length-corrected-kinematic comparison (collapse-to-geometry) and, once a probe tier exists, the ∇V ablation (collapse-to-V2).

The coasting set is bootstrapped and mildly circular (V3 report § coasting). On the constructive curriculum, prefer the construction-time off-path label as the coasting set — it needs no bootstrap and breaks the circularity. Do not let a provisional-score-chosen coasting set define the anchors the final score then “discovers.”

6 Part E — Later tiers (typed stubs)

Not implemented. Each is a named consumer of the Part A contract; logic is written only when Tier 1’s outcome selects it (V3 report § tiers).

probe — off-path ∇V (fix/improvement)

Consumes dense `hidden` at `layer` and upper-layer stack (provisioned in Part A). Enables Test 2, Branch C, and the full impulse residual. *The load-bearing engineering seam* (partial-forward-from-layer with cache reuse; model/version-specific). Stub: interface only.

Branch B drag refinement / Branch C gauge / Branch D Jacobi / Branch E GENERIC

Consume velocity, V , and (C/D) the probe Jacobian. Entered per the test→branch routing. Stub: interface only.

first-content-token early decision (capability improvement)

Consumes `window_flags` and the per-span first-content-token index; produces the lead-time/accuracy curve. Expected to *cost* accuracy for lead time. Stub: interface only.

NLI cross-check (optional, small models)

Consumes spans; an alternative adjudicator, fixed per label set if used. Stub: interface only.

Bottom line. This document is decision-complete for Tier 0, the data contract, and Tier 1: an implementer can build them without inventing a choice the document should have made. It is deliberately incomplete past Tier 1 — not an omission but the same fail-cheap discipline as the experiment tiering. The next real information is the Tier 1 coasting-fit (Test 1) on real traces; no further planning substitutes for it.